

Improving Medical Calculation of LLMs with Embedded Coding

Tianshi Ming

tianshim@andrew.cmu.edu
Carnegie Mellon University

Yingying Zhang

ninzhang@tencent.com
Tencent YouTu Lab

Xian Wu

kevinxwu@tencent.com
Tencent YouTu Lab

Abstract

Large Language Models (LLMs) perform well on medical examinations and question-answering benchmarks, but remain unreliable on medical calculation tasks that require exact numerical outputs. These calculations support high-stakes decisions such as medication dosing, organ-function assessment, and prognostic scoring, for which even small errors can have serious clinical consequences. We introduce MedCode, a framework that improves medical calculation by training LLMs to generate embedded executable code. Given a clinical context, the model identifies the relevant calculator, extracts its input variables, and produces a script that delegates arithmetic operations to a deterministic interpreter. Executing the script returns the calculated value together with an explanation and the appropriate unit. We construct supervised fine-tuning (SFT) and preference datasets from the MedCalc benchmark and additionally curate a dataset for calculation tasks in Intensive Care Unit (ICU) scenarios. We further propose weighted Direct Preference Optimization (wDPO), which adaptively emphasizes preference pairs that are difficult for the model to distinguish. Experiments with LLaMA3-8B, Qwen2.5-7B, and Mistral-7B show absolute accuracy gains of 20–30 percentage points, demonstrating the effectiveness of embedded code generation for medical calculation.

1 Introduction

Large Language Models (LLMs), including GPT-4 (Achiam et al., 2023), have demonstrated strong contextual understanding, attained high performance on medical examinations (Singhal et al., 2025), and shown potential in clinical applications such as disease diagnosis (Gao et al., 2025) and treatment planning (Zhou et al., 2023b; Stade et al., 2024). Nevertheless, reliable numerical calculation remains a substantial challenge. Even when a model can recall relevant medical knowledge,

it may fail to apply that knowledge correctly in quantitative reasoning (Zhang et al., 2025a). This limitation constrains the use of LLMs in clinical settings that require precise numerical outputs.

This limitation is especially consequential in the Intensive Care Unit (ICU), where clinical decisions frequently depend on timely and accurate calculations. For example, computing the Glomerular Filtration Rate (GFR), an indicator of renal function, requires selecting the appropriate equation and extracting variables such as serum creatinine and age from a patient note. As illustrated in Figure 1, GPT-4 omits coefficients required by the specified calculator, while Mistral-7B exhibits both formula-selection and arithmetic errors. Such failures may propagate to downstream decisions, including medication dosing and prognostic assessment. Consistent with the findings of MedCalc-Bench (Khandekar et al., 2024), these examples highlight the need for methods that improve numerical reliability rather than relying on direct text generation alone.

Our analysis suggests that many failures arise after the relevant information has been identified: models may extract the required variables correctly but make errors in multi-step arithmetic or in the constants and conditions of a medical formula. This observation motivates separating contextual interpretation from numerical computation. The LLM can parse the clinical context and instantiate the required calculation, while a deterministic interpreter performs the arithmetic.

Motivated by this separation, we propose MedCode. Rather than directly generating a numerical answer, the model is fine-tuned to produce an executable embedded code script that combines calculation logic with textual output. As shown in Figure 1, arithmetic operations are expressed as code, while explanations and units are emitted through `print()` statements. A single script execution therefore returns both the numerical result and its accompanying explanation. MedCode requires

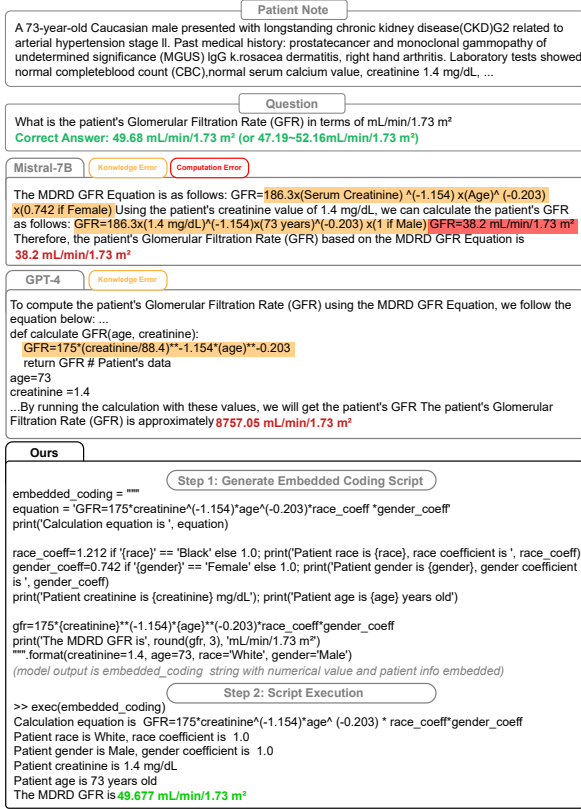


Figure 1: An example medical calculation task. Mistral-7B and GPT-4 produce incorrect answers because of formula-selection or arithmetic errors. MedCode instead generates an embedded code script and executes it with a deterministic interpreter. The execution output contains both the calculation procedure and the numerical answer. This pipeline requires one LLM forward pass for script generation, followed by code execution without an additional model call.

one LLM decoding pass followed by deterministic execution, avoiding the repeated model calls used in multi-stage function-calling pipelines (Shen, 2024).

To train models for this generation format, we convert MedCalc-Bench examples into executable scripts for supervised fine-tuning (SFT). We then collect erroneous scripts produced by the fine-tuned model and pair each with its correct reference script, yielding preference data without requiring separate pairwise annotation. We additionally construct a clinician-authored synthetic dataset for ICU scenarios, where a model must extract, filter, and count longitudinal clinical measurements.

Building on these datasets, we introduce weighted Direct Preference Optimization (wDPO). Standard preference optimization treats all response pairs uniformly, although pairs that differ only in an extracted argument, coefficient, or unit

conversion can be more difficult to distinguish than pairs containing an obvious structural error. wDPO assigns adaptive weights to the preference loss according to the probability gap between the preferred and dispreferred scripts, thereby emphasizing ambiguous pairs during training.

The contributions of this paper are as follows:

- We propose MedCode, an embedded code generation framework that improves medical calculation accuracy by 20–30 absolute percentage points.
- We introduce an error-driven procedure for constructing preference pairs and propose wDPO to optimize a code-generating model with difficulty-aware preference weights.
- We evaluate MedCode on the public MedCalc benchmark and a newly constructed ICU calculation dataset. The datasets and code will be released upon publication.

2 Related Work

2.1 Medical Large Language Model

Medical large language models (Med-LLMs) have been developed for a broad range of clinical and biomedical tasks (Chen et al., 2024; Saab et al., 2024; Luo et al., 2024; Gao et al., 2023b; Singhal et al., 2023). However, medical metric calculation requires a distinct combination of domain knowledge, information extraction, and exact numerical computation. Common benchmarks, including PubMedQA (Jin et al., 2019), MedQA (Jin et al., 2020), MedMCQA (Pal et al., 2022), and MMLU (Hendrycks et al., 2021), primarily use multiple-choice questions and therefore do not directly evaluate open-ended calculation. MedCalc-Bench (Khandekar et al., 2024) instead requires models to produce answers for equation-based and rule-based medical calculators without predefined options, and reports low accuracy for both open- and closed-source models. MedCode addresses this setting by combining domain-specific supervised fine-tuning with preference learning for executable medical calculation.

2.2 Coding Generation

Code-assisted reasoning has been widely studied as a means of reducing computational errors in LLM outputs (Zhang et al., 2025b). Verification-based methods such as CSV (Zhou et al., 2023a)

use an interpreter to check generated answers, whereas ToRA (Gou et al., 2024) interleaves natural-language reasoning and code execution. Because execution results are fed back into subsequent reasoning steps, these methods may require multiple model calls. MedCode instead trains the model to emit a complete executable script in one decoding pass, followed by deterministic execution.

Program-aided methods directly generate code whose execution supplies the answer. PAL (Gao et al., 2023a) and PoT (Chen et al., 2023) combine intermediate reasoning with program generation, while Chain of Code (Li et al., 2024) simulates line-by-line code execution. Medical calculation differs from general-purpose program synthesis because the generated program must encode the correct clinical calculator, units, coefficients, and input variables. We therefore construct domain-specific code training data rather than relying exclusively on inference-time prompting. Our experiments show that models fine-tuned on these data can outperform substantially larger models that receive only prompt-based code demonstrations.

Agent-based systems such as AgentMD (Jin et al., 2024), EHRAgent (Shi et al., 2024), TOOL-MAKER (Wölflin et al., 2025), and REFLECT-TOOL (Liao et al., 2025) coordinate tool selection, execution, and verification across multiple inference stages. These frameworks provide broad tool-use capabilities but incur additional model calls and are not designed specifically for medical metric calculation. By contrast, MedCode encodes calculator knowledge in its training data and generates the complete calculation program in a single model pass. The resulting script is then executed without further agent orchestration.

3 Dataset Generation

3.1 Embedded Coding Dataset Generation based on MedCalc

MedCalc-Bench (Khandekar et al., 2024) comprises 55 commonly used medical calculation tasks collected from online medical calculators¹. Each example contains a patient note, a calculation question, and a manually reviewed reference answer (Figure 2). Solving an example requires selecting the relevant calculator and applying it to clinical variables described in the note.

¹<https://www.mdcalc.com/#Popular>

We use code as an intermediate representation and delegate numerical operations to an executable program. Based on the MedCalc calculators, we construct an embedded code dataset with three components. **(1) Medical knowledge:** code comments specify the calculator logic and formula. **(2) Embedded computation:** a compact script instantiates the formula with values extracted from the note and applies explicit conversion factors when units differ. **(3) Output generation:** the script prints the result together with an explanation and the corresponding unit. Appendix A details the script designs.

3.2 Preference Dataset Generation Based on MedCalc

We first fine-tune the model on the embedded code dataset. Although SFT substantially improves accuracy, the resulting model can still extract an incorrect argument or generate an erroneous expression. Such failures often differ from the reference script by only a coefficient, variable, or implementation detail. We therefore introduce preference learning to train the model to distinguish correct scripts from these plausible but incorrect alternatives.

Because explicitly annotated preference data for medical calculation code are scarce, we construct preference pairs from errors produced by the SFT model, as illustrated in Figure 2. The procedure consists of the following two steps.

Inference on training data We sample k instances from the SFT dataset described in Section 3.1, preserving the original distribution over equation-based and rule-based tasks. A new subset is sampled for each inference run. The fine-tuned model then generates scripts for these instances at temperatures from 0 to 1 in increments of 0.1. Varying the temperature increases output diversity and exposes a broader set of failure modes. Appendix B provides the sampling and inference settings.

Pairing error scripts with correct scripts We execute the generated scripts and retain outputs that are incorrect or fail to execute. Each erroneous script is paired with the reference implementation for the same input, yielding a dispreferred response and a preferred response. This error-driven construction produces preference pairs without requiring an independently annotated preference corpus.

3.3 ICU-Dataset

MedCalc-Bench evaluates calculators applied to a single summarized patient state. ICU work-

> Equation-based Calculator Example: Modification of Diet in Renal Disease Glomerular Filtration Rate (MDRD GFR)	
Patient Note	A 73-year-old Caucasian male presented with longstanding chronic kidney disease (CKD) G2 related to arterial hypertension stage II. Past medical history: prostate cancer and monoclonal gammopathy of undetermined significance (MGUS) IgG k, rosacea dermatitis, right hand arthritis. ...
Question	What is the patient’s Glomerular Filtration Rate (GFR) in terms of mL/min/1.73 m ² ?
Generated Code	<pre># GFR = 175 * creatinine^(-1.154) * age^(-0.203) * race_coefficient * gender_coefficient race_coefficient = 1.212 if 'Black' == 'Black' else 1.0 gender_coefficient = 0.742 if 'Male' == 'Female' else 1.0 gfr = 175 * 10.6**(-1.154) * 49**(-0.203) * race_coefficient * gender_coefficient gfr = round(gfr, 3) print('The MDRD GFR is ', gfr, ' mL/min/1.73 m^2')</pre>
Code Execution Result	The MDRD GFR is 49.677 mL/min/1.73 m ²
> Rule-based Calculator Example: Centor Score (Modified/McIsaac) for Strep Pharyngitis	
Patient Note	A 15-year-old female presents to your office for evaluation of a painful neck mass. The family reports the patient has had a midline neck mass for many years, but had become painful over the past week with erythema at the overlying skin. ...
Question	What is the patient’s Centor Score?
Generated Code	<pre># Centor Score (Modified/McIsaac) criteria: age, temperature, ... # age thresholds are: (45, 'inf'), (3, 15) ... # The patient has tender/swollen anterior cervical lymph nodes or not score = 0 score += -1 * (45 <= 15 < float('inf')) + 1 * (3 <= 15 < 15) ... has_exudate_swelling_tonsils = False score += 1 if has_exudate_swelling_tonsils else 0 print('The Centor Score (Modified/McIsaac) for Strep Pharyngitis is', score)</pre>
Code Execution Result	Centor Score (Modified/McIsaac) for Strep Pharyngitis is 2

Table 1: Examples of embedded code generation for equation-based and rule-based medical calculation tasks.

flows, by contrast, often require operations over measurements recorded at multiple time points. The resulting tasks combine information extraction, threshold-based filtering, and counting over longitudinal Electronic Health Records (EHRs), placing simultaneous demands on contextual understanding and numerical reliability.

We therefore construct a dataset for medical statistical calculation in ICU scenarios. Each instance provides longitudinal EHR entries and a threshold-based criterion; the model must identify the entries satisfying the criterion and report their number. We evaluate two aspects of performance: **(1) extraction accuracy**, the proportion of relevant records that are correctly extracted, and **(2) frequency counting**, the accuracy of the final count. The dataset is written and verified by experienced clinicians.

Data Format The ICU dataset is manually constructed and verified by experienced clinicians. Each synthetic example contains basic patient information and a set of timestamped EHR entries, including metric names and values. An SFT instance consists of an instruction, the EHR entries, and a compact reference script. Given a threshold or interval, the model must identify and count

the entries satisfying the specified condition. The dataset is written in simplified Chinese and contains 5,000 training examples and 1,000 test examples. Appendix E presents a translated example.

Data Augmentation We augment the dataset by recombining existing entries and sampling new values within clinically plausible ranges. Table 12 summarizes the clinical categories, metrics, units, and value ranges. The resulting dataset covers 7 clinical categories and 16 medical metrics.

4 Preference Training Strategy

Supervised fine-tuning substantially improves calculation accuracy, but the model continues to produce subtle script errors, including incorrect coefficients and unit conversions. We therefore apply preference optimization to distinguish reference scripts from erroneous alternatives.

4.1 Weighted Direct Preference Optimization

Preference learning trains the model to distinguish a preferred script y_w from a dispreferred script y_l for input x , as illustrated in Figure 2. Direct Preference Optimization (DPO) (Rafailov et al., 2024) compares the likelihood ratios of these responses under the policy π_θ and a fixed reference model

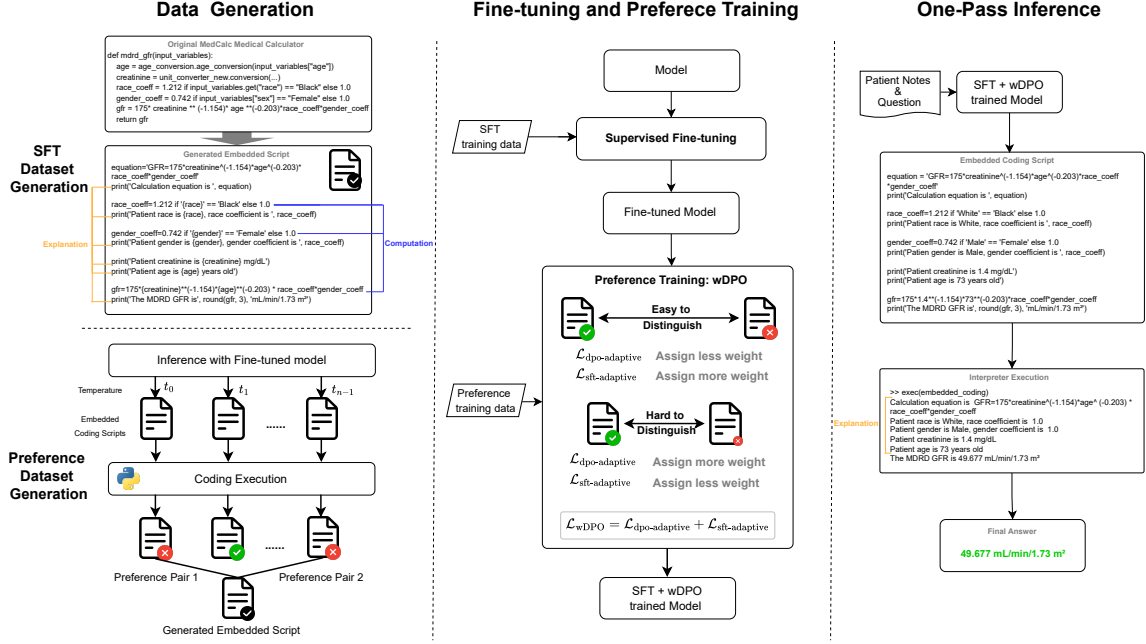


Figure 2: Overview of MedCode. **1) Data generation:** We convert MedCalc calculators into an embedded code dataset for SFT and pair erroneous scripts produced by the fine-tuned model with reference scripts to construct preference data. **2) Fine-tuning and preference training:** We first perform SFT and then apply wDPO, which adaptively weights the preference objective. **3) One-pass inference:** The model generates a script containing values extracted from the patient note, and a deterministic interpreter executes the script to produce the final answer and calculation explanation.

π_{ref} . Its objective is given in Equation 1.

$$\begin{aligned} \mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) &= -\mathbb{E}_{(x,y) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right] \\ &= -\mathbb{E}_{(x,y) \sim \mathcal{D}} [\log \sigma (\beta \cdot \hat{r}(x, y_w) - \beta \cdot \hat{r}(x, y_l))]. \end{aligned} \quad (1)$$

where $\hat{r}(x, y_w)$ and $\hat{r}(x, y_l)$ denote the implicit rewards assigned to the preferred and dispreferred scripts, respectively. Standard DPO assigns equal importance to every preference pair, although the pairs vary in difficulty. For example, a missing computation step creates a conspicuous structural difference, whereas an incorrect unit-conversion factor may differ from the reference by only a single constant. Appendix C provides examples of these minor and substantial errors.

We address this variation with an adaptive preference objective that weights each pair according to how clearly the current model separates its two responses. Let $w(\pi_{\theta}(y_w|x), \pi_{\theta}(y_l|x))$ denote this weight. Pairs with similar model probabilities receive larger weights, whereas pairs with a clear probability gap receive smaller weights. The resulting objective $\mathcal{L}_{\text{DPO-adaptive}}$ is defined in Equa-

tion 2.

$$\begin{aligned} \mathcal{L}_{\text{DPO-adaptive}}(\pi_{\theta}; \pi_{\text{ref}}) &= -\mathbb{E}_{(x,y) \sim \mathcal{D}} w(\pi_{\theta}(y_w|x), \pi_{\theta}(y_l|x)) [\log \sigma (\beta \Delta \hat{r})]. \end{aligned} \quad (2)$$

where $\Delta \hat{r} = \hat{r}(x, y_w) - \hat{r}(x, y_l)$. We use the normalized difference between $\pi_{\theta}(y_w|x)$ and $\pi_{\theta}(y_l|x)$ as the model-dependent measure of pairwise separation. The adaptive weight is defined as follows:

$$w(\pi_{\theta}(y_w|x), \pi_{\theta}(y_l|x)) = 1 - \frac{|\pi_{\theta}(y_w|x) - \pi_{\theta}(y_l|x)|}{|\pi_{\theta}(y_w|x)| + |\pi_{\theta}(y_l|x)|}. \quad (3)$$

The denominator applies an L_1 normalization to keep the probability difference on a common scale. Taking the absolute value makes the weight depend on the magnitude of separation, including cases in which the current model ranks y_l above y_w . The probabilities used to compute w are detached from the computation graph, so gradients do not propagate through the weight. We recompute these probabilities after each training epoch.

The adaptive DPO term emphasizes pairs whose responses are difficult to separate. Conversely, pairs with a large probability gap receive a smaller preference weight and may contribute less to training. To retain direct supervision from their preferred responses, we add an SFT objective with the

complementary coefficient $1 - w$. The adaptive SFT term is defined in Equation 4.

$$\begin{aligned} \mathcal{L}_{\text{sft-adaptive}}(\pi_\theta; \pi_{\text{ref}}) \\ = -\mathbb{E}_{(x,y) \sim \mathcal{D}} (1 - w(\pi_\theta(y_w|x), \pi_\theta(y_l|x))) [\log \pi_\theta(y_w|x)]. \end{aligned} \quad (4)$$

When w is small, $\mathcal{L}_{\text{sft-adaptive}}$ places greater weight on maximizing the likelihood of the preferred response. We define the complete wDPO objective as the sum of the adaptive DPO and adaptive SFT terms in Equation 5.

$$\begin{aligned} \mathcal{L}_{\text{wDPO}}(\pi_\theta; \pi_{\text{ref}}) = \\ \mathcal{L}_{\text{dpo-adaptive}}(\pi_\theta; \pi_{\text{ref}}) + \mathcal{L}_{\text{sft-adaptive}}(\pi_\theta; \pi_{\text{ref}}). \end{aligned} \quad (5)$$

5 Experiments

5.1 Experiment Settings

We evaluate LLaMA3-8B-Instruct (Grattafiori et al., 2024), Mistral-7B-Instruct (Jiang et al., 2023), Qwen2.5-7B-Instruct (Qwen et al., 2025), and Baichuan2-7B-Chat (Yang et al., 2023); GPT-3.5 and GPT-4 are included as prompted baselines. For the inference-time baselines, we follow the prompts and inference settings released with MedCalc-Bench (Khandekar et al., 2024). We implement SFT and preference optimization with LLaMA-Factory (Zheng et al., 2024). Appendix B reports the complete training, inference, and prompt configurations.

5.2 Main Results

We first evaluate MedCode on MedCalc-Bench, which contains four equation-based and three rule-based task categories. Its training split comprises 9,766 examples with patient notes, questions, reference answers, and the medical entities required by each calculator. Answers include continuous values, discrete scores, dates, and time intervals. We compare inference-time prompting (zero-shot, one-shot, and Chain-of-Thought) with three training-based variants: direct-answer SFT (SFT Direct), embedded-code SFT (SFT Code), and SFT followed by wDPO (SFT Code+wDPO). Table 2 reports the results.

Training-based methods substantially outperform inference-time prompting for the same backbone. For LLaMA3-8B, average accuracy increases from 27.13 with one-shot CoT to 33.37 with SFT Direct and 54.39 with SFT Code, corresponding to gains of 6.24 and 27.26 percentage points. For Mistral-7B, the corresponding scores are 16.05, 37.34, and 57.92, yielding gains of 21.29

and 41.87 points. Adding wDPO further raises average accuracy by 0.95 points for LLaMA3-8B and 1.81 points for Mistral-7B. Both SFT-Code models also exceed the reported GPT-4 averages.

The gains from wDPO are concentrated in specific task categories. Relative to SFT Code, it improves laboratory-task accuracy by 5.52 points for LLaMA3-8B and 5.83 points for Mistral-7B. On LLaMA3-8B, the dosage and diagnosis categories improve by 22.50 and 8.33 points, respectively, although several other categories remain unchanged or decrease. These results indicate that approximately 1,500 mined preference pairs can improve selected fine-grained behaviors, while also showing that the benefit is not uniform across tasks.

5.3 Evaluation on ICU dataset

We next evaluate MedCode on the ICU dataset (Table 3). For LLaMA3-8B, replacing direct-answer SFT with embedded-code SFT increases extraction accuracy from 52.39 to 82.10 and frequency-count accuracy from 75.10 to 97.60, gains of 29.71 and 22.50 percentage points. wDPO provides additional gains to 83.20 and 98.00. Qwen2.5-7B shows the same overall pattern, with a particularly large improvement in frequency counting. These results support the use of executable code for threshold-based extraction and aggregation.

We additionally compare MedCode with reasoning-focused LLMs on the ICU dataset (Table 4). LLaMA3-8B-Instruct trained with SFT and wDPO reaches 83.20 extraction accuracy and 98.00 frequency-count accuracy, compared with 85.87 and 99.53 for DeepSeek-R1. Although DeepSeek-R1 and OpenAI-o3 obtain higher scores, the smaller fine-tuned model remains competitive on this task. This comparison is relevant to clinical deployment, where restrictions on sending data to external APIs and limited local compute can constrain the use of proprietary services or very large models.

5.4 Ablation Study on Adaptive SFT

We study the complementary SFT term introduced in Section 4.1 by comparing DPO and wDPO with fixed or adaptive regularization (Table 7). Adding a fixed SFT term improves both DPO and wDPO over their unregularized variants. For wDPO, adaptive regularization achieves the highest equation-based accuracy (72.26) and overall average (55.34), while a fixed coefficient of 1.0 yields the highest rule-based score (29.40). Thus, adaptive regular-

Model	Equation-based				Rule-based			Avg.
	Lab	Phys.	Date	Dosage	Risk	Sev.	Diag	
Zero-shot								
LLaMA3-8B	10.70	19.17	3.33	5.00	12.50	8.75	25.00	13.09
Mistral-7B	10.70	18.33	3.33	0.00	4.58	3.75	13.33	9.8
GPT-3.5	17.13	35.00	13.33	5.00	12.92	6.25	18.33	18.82
GPT-4	14.37	34.58	38.33	15.00	14.58	15.00	20.00	20.82
Zero-shot CoT								
LLaMA3-8B	16.51	25.00	1.67	7.50	11.25	13.75	26.67	16.43
Mistral-7B	10.09	14.58	1.67	0.00	9.58	7.50	25.00	10.79
GPT-3.5	20.49	45.00	11.67	17.50	13.33	10.00	31.67	23.69
GPT-4	26.30	71.25	48.33	40.00	27.50	15.00	28.33	37.92
One-shot CoT								
LLaMA3-8B	34.86	35.42	3.33	2.50	20.00	11.25	41.67	27.13
Mistral-7B	11.01	30.42	6.67	0.00	16.25	6.25	18.33	16.05
GPT-3.5	30.89	59.17	41.67	15.00	23.33	17.50	35.00	34.86
GPT-4	51.68	77.50	46.67	37.50	33.75	27.50	53.33	50.91
SFT Direct								
LLaMA3-8B	19.27	85.00	33.33	28.57	11.25	10.00	41.67	33.37
Mistral-7B	19.27	91.25	31.67	17.50	18.75	15.00	43.33	37.34
SFT Code								
LLaMA3-8B	66.26	92.53	26.67	42.50	23.24	15.00	50.00	54.39
Mistral-7B	70.55	99.59	33.33	45.00	23.24	18.75	46.67	57.92
GPT-3.5*	32.21	34.02	36.67	15.00	4.15	7.50	13.33	22.81
GPT-4*	57.67	90.46	36.67	45.00	23.24	11.25	97.50	52.86
SFT Code+wDPO								
LLaMA3-8B	71.78	83.82	33.33	65.00	22.82	10.00	58.33	55.34
Mistral-7B	76.38	99.59	33.33	45.00	23.24	18.75	46.67	59.73

Table 2: Results on MedCalc-Bench. We evaluate LLaMA3-8B-Instruct, Mistral-7B-Instruct-v0.2, GPT-3.5-Turbo, and GPT-4. CoT denotes Chain-of-Thought prompting. Zero-shot, zero-shot CoT, and one-shot CoT results are taken from MedCalc-Bench (Khandekar et al., 2024). *For code generation with GPT-3.5 and GPT-4, we use one in-context example.

Model	Methods	extr.	fcnt.
LLaMA3-8B	One-shot Direct	39.22	0.0
	SFT-Direct	52.39	75.10
	SFT-Code	82.10	97.60
	SFT-Code-wDPO	83.20	98.00
Qwen2.5-7B	One-shot Direct	47.90	0.0
	SFT-Direct	56.70	77.90
	SFT-Code	59.40	97.00
	SFT-Code-wDPO	62.10	97.50

Table 3: Results on the ICU dataset. “extr.” denotes extraction accuracy and “fcnt.” denotes frequency-count accuracy. We evaluate LLaMA3-8B-Instruct and Qwen2.5-7B-Instruct.

ization provides the best aggregate result, but its advantage is not uniform across task types.

5.5 Comparison with zero and few-shot

We compare zero-shot and few-shot configurations to measure the effect of adding in-context demon-

Model	extr.	fcnt.
LLaMA3-8B-Instruct (wDPO)	83.20	98.00
Qwen2.5-7B-Instruct (wDPO)	62.10	97.50
Baichuan2-7B-Chat (wDPO)	79.13	96.54
DeepSeek-R1	85.87	99.53
OpenAI-o3	90.10	99.10

Table 4: Comparison with reasoning-focused models on the ICU dataset. “extr.” denotes extraction accuracy and “fcnt.” denotes frequency-count accuracy.

strations to the code-generation pipeline. As shown in Table 5, increasing the number of demonstrations from zero to one and three raises the average score from 33.37 to 34.56 and 36.77, respectively. The improvement is consistent but modest relative to the gains from training-based code generation reported in Table 2, indicating that the main benefit arises from task-specific training rather than from additional inference-time examples.

Strategy	Lab	Phys.	Date	Dos.	Risk	Sev.	Diag.	Avg.
Zero-shot	10.70	19.17	3.33	5.00	12.50	8.75	25.00	13.09
Zero-shot CoT	16.51	25.00	1.67	7.50	11.25	13.75	26.67	16.43
One-shot CoT	34.86	35.42	3.33	2.50	20.00	11.25	41.67	27.13
Zero-shot SFT Code	19.27	85.00	33.33	28.57	11.25	10.00	41.67	33.37
One-shot SFT Code	20.38	89.85	35.25	30.24	11.92	10.59	44.08	34.56
3-shot SFT Code	21.49	94.72	37.17	31.89	12.55	11.15	46.49	36.77

Table 5: Comparison of zero and few-shot strategies.

Method	Lab	Phys.	Date	Dosage	Risk	Sev.	Diag.	Avg.
rDPO (Chowdhury et al., 2024)	68.45	79.20	30.12	59.33	25.10	12.55	51.33	50.84
ROPO (Liang et al., 2024)	70.80	82.50	34.00	63.75	23.90	11.30	53.45	54.25
WPO (Zhou et al., 2024)	69.20	81.00	31.45	61.25	24.70	11.00	47.70	52.33
wDPO (ours)	71.78	83.82	33.33	65.00	22.82	10.00	58.33	55.34

Table 6: Comparison of preference optimization methods on MedCalc-Bench using LLaMA3-8B-Instruct. All methods use the same SFT-Code initialization and preference dataset.

Loss	Reg.	Eq.	Rl.	Avg.
$\mathcal{L}_{\text{SFT}}$	-	70.77	25.72	54.39
$\mathcal{L}_{\text{DPO}}$	-	60.17	5.23	41.36
	$0.5\mathcal{L}_{\text{SFT}}$	62.82	8.66	43.13
	$1.0\mathcal{L}_{\text{SFT}}$	66.87	10.24	46.28
$\mathcal{L}_{\text{wDPO}}$	-	66.50	15.37	47.68
	$0.5\mathcal{L}_{\text{SFT}}$	67.32	20.27	52.17
	$1.0\mathcal{L}_{\text{SFT}}$	68.36	29.40	54.20
	adpt.	72.26	25.72	55.34

Table 7: Results of regularization strategies on MedCalc-Bench using LLaMA3-8B-Instruct. ‘adpt.’ denotes the adaptive coefficient; **Eq.** and **Rl.** denote equation-based and rule-based tasks; ‘-’ indicates that no additional SFT regularization is used.

5.6 Comparison with Other Preference Optimization Approaches

We compare wDPO with three representative preference optimization methods: rDPO (Chowdhury et al., 2024), ROPO (Liang et al., 2024), and WPO (Zhou et al., 2024).

All methods start from the same LLaMA3-8B-Instruct SFT-Code checkpoint and use the same MedCalc-Bench preference data. As shown in Table 6, their average accuracies are 50.84 for rDPO, 54.25 for ROPO, 52.33 for WPO, and 55.34 for wDPO. Thus, wDPO performs best among the compared preference methods and improves over the SFT-Code baseline of 54.39 by 0.95 percentage points; the other preference methods do not consistently exceed that baseline. wDPO also obtains the highest aggregate performance over the four equation-based categories, supporting the value of adaptive weighting for subtle numerical differences.

6 Conclusion

We introduce MedCode, a framework for improving LLM-based medical calculation through embedded code generation. Rather than producing a numerical answer directly, the fine-tuned model generates an executable script that instantiates the relevant medical calculator with values extracted from the input. A deterministic interpreter then performs the arithmetic and returns the result with its explanation and unit.

We construct an executable-code version of MedCalc-Bench and a synthetic dataset for longitudinal ICU calculation. In addition to SFT, we mine erroneous model-generated scripts to form preference pairs and introduce wDPO to weight these pairs according to their difficulty. Across the evaluated models and datasets, embedded code generation substantially improves calculation accuracy, while wDPO provides additional gains on selected tasks.

Limitations

MedCode relies on predefined medical calculators and therefore may not generalize to formulas or clinical rules absent from the training data. The error-mining procedure is also limited by the failure modes produced by the SFT model and may under-represent rare or clinically complex errors. Finally, our experiments focus on short, well-specified calculation tasks; performance in longer clinical workflows, across multiple patients, or under incomplete and conflicting records remains untested.

Ethics

This work follows the ACL Code of Ethics. MedCalc-Bench is publicly available, and the ICU dataset consists of synthetic records manually authored and verified by experienced clinicians; it contains no real patient records or personally identifiable information. The datasets and models are intended for research on calculation reliability and have not been validated for autonomous clinical decision-making.

References

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, and 1 others. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Junying Chen, Xidong Wang, Ke Ji, Anningzhe Gao, Feng Jiang, Shunian Chen, Hongbo Zhang, Dingjie Song, Wenya Xie, Chuyi Kong, Jianquan Li, Xiang Wan, Haizhou Li, and Benyou Wang. 2024. [Huatuogpt-ii, one-stage training for medical adaptation of llms](#). *Preprint*, arXiv:2311.09774.
- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. 2023. [Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks](#). *Preprint*, arXiv:2211.12588.
- Sayak Ray Chowdhury, Anush Kini, and Nagarajan Natarajan. 2024. [Provably robust dpo: Aligning language models with noisy feedback](#). *Preprint*, arXiv:2403.00409.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023a. [Pal: Program-aided language models](#). *Preprint*, arXiv:2211.10435.
- Weihao Gao, Zhuo Deng, Zhiyuan Niu, Fujun Rong, Chucheng Chen, Zheng Gong, Wenze Zhang, Daimin Xiao, Fang Li, Zhenjie Cao, Zhaoyi Ma, Wenbin Wei, and Lan Ma. 2023b. [Ophglm: Training an ophthalmology large language-and-vision assistant based on instructions and dialogue](#). *Preprint*, arXiv:2306.12174.
- Yanjun Gao, Ruizhe Li, Emma Croxford, John Caskey, Brian W Patterson, Matthew Churpek, Timothy Miller, Dmitriy Dligach, and Majid Afshar. 2025. Leveraging medical knowledge graphs into large language models for diagnosis prediction: Design and application study. *JMIR AI*, 4:e58670.
- Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujia Yang, Minlie Huang, Nan Duan, and Weizhu Chen. 2024. [Tora: A tool-integrated reasoning agent for mathematical problem solving](#). *Preprint*, arXiv:2309.17452.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, and 12 others. 2024. [The llama 3 herd of models](#). *Preprint*, arXiv:2407.21783.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. [Measuring massive multitask language understanding](#). *Preprint*, arXiv:2009.03300.
- Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. 2023. [Mistral 7b](#). *Preprint*, arXiv:2310.06825.
- Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. 2020. [What disease does this patient have? a large-scale open domain question answering dataset from medical exams](#). *Preprint*, arXiv:2009.13081.
- Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William Cohen, and Xinghua Lu. 2019. [PubMedQA: A dataset for biomedical research question answering](#). In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2567–2577, Hong Kong, China. Association for Computational Linguistics.
- Qiao Jin, Zhizheng Wang, Yifan Yang, Qingqing Zhu, Donald Wright, Thomas Huang, W John Wilbur, Zhe He, Andrew Taylor, Qingyu Chen, and Zhiyong Lu. 2024. [Agentmd: Empowering language agents for risk prediction with large-scale clinical tool learning](#). *Preprint*, arXiv:2402.13225.
- Nikhil Khandekar, Qiao Jin, Guangzhi Xiong, Soren Dunn, Serina S Applebaum, Zain Anwar, Maame Sarfo-Gyamfi, Conrad W Safranek, Abid A Anwar, Andrew Zhang, Aidan Gilson, Maxwell B Singer, Amisha Dave, Andrew Taylor, Aidong Zhang, Qingyu Chen, and Zhiyong Lu. 2024. [Medcalc-bench: Evaluating large language models for medical calculations](#). In *Advances in Neural Information Processing Systems*, volume 37, pages 84730–84745. Curran Associates, Inc.
- Chengshu Li, Jacky Liang, Andy Zeng, Xinyun Chen, Karol Hausman, Dorsa Sadigh, Sergey Levine, Li Fei-Fei, Fei Xia, and Brian Ichter. 2024. [Chain of code: Reasoning with a language model-augmented code emulator](#). *Preprint*, arXiv:2312.04474.
- Xize Liang, Chao Chen, Shuang Qiu, Jie Wang, Yue Wu, Zhihang Fu, Zhihao Shi, Feng Wu, and Jieping

- Ye. 2024. [Ropo: Robust preference optimization for large language models](#). *Preprint*, arXiv:2404.04102.
- Yusheng Liao, Shuyang Jiang, Yanfeng Wang, and Yu Wang. 2025. [Reflectool: Towards reflection-aware tool-augmented clinical agents](#). *Preprint*, arXiv:2410.17657.
- Ling Luo, Jinzhong Ning, Yingwen Zhao, Zhijun Wang, Zeyuan Ding, Peng Chen, Weiru Fu, Qinyu Han, Guangtao Xu, Yunzhi Qiu, Dinghao Pan, Jiru Li, Hao Li, Wenduo Feng, Senbo Tu, Yuqi Liu, Zhihao Yang, Jian Wang, Yuanyuan Sun, and Hongfei Lin. 2024. [Taiyi: a bilingual fine-tuned large language model for diverse biomedical tasks](#). *Journal of the American Medical Informatics Association*, 31(9):1865–1874.
- Ankit Pal, Logesh Kumar Umapathi, and Malaikannan Sankarasubbu. 2022. [Medmcqa: A large-scale multi-subject multi-choice dataset for medical domain question answering](#). In *Proceedings of the Conference on Health, Inference, and Learning*, volume 174 of *Proceedings of Machine Learning Research*, pages 248–260. PMLR.
- Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, and 25 others. 2025. [Qwen2.5 technical report](#). *Preprint*, arXiv:2412.15115.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2024. Direct preference optimization: your language model is secretly a reward model. In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23*, Red Hook, NY, USA. Curran Associates Inc.
- Khaled Saab, Tao Tu, Wei-Hung Weng, Ryutaro Tanno, David Stutz, Ellery Wulczyn, Fan Zhang, Tim Strother, Chunjong Park, Elahe Vedadi, Juanma Zambrano Chaves, Szu-Yeu Hu, Mike Schaeckermann, Aishwarya Kamath, Yong Cheng, David G. T. Barrett, Cathy Cheung, Basil Mustafa, Anil Palepu, and 23 others. 2024. [Capabilities of gemini models in medicine](#). *Preprint*, arXiv:2404.18416.
- Zhuocheng Shen. 2024. Llm with tools: A survey. *arXiv preprint arXiv:2409.18807*.
- Wenqi Shi, Ran Xu, Yuchen Zhuang, Yue Yu, Jieyu Zhang, Hang Wu, Yuanda Zhu, Joyce Ho, Carl Yang, and May D. Wang. 2024. [Ehrgent: Code empowers large language models for few-shot complex tabular reasoning on electronic health records](#). *Preprint*, arXiv:2401.07128.
- Karan Singhal, Tao Tu, Juraj Gottweis, Rory Sayres, Ellery Wulczyn, Mohamed Amin, Le Hou, Kevin Clark, Stephen R Pfohl, Heather Cole-Lewis, and 1 others. 2025. Toward expert-level medical question answering with large language models. *Nature Medicine*, pages 1–8.
- Karan Singhal, Tao Tu, Juraj Gottweis, Rory Sayres, Ellery Wulczyn, Le Hou, Kevin Clark, Stephen Pfohl, Heather Cole-Lewis, Darlene Neal, Mike Schaeckermann, Amy Wang, Mohamed Amin, Sami Lachgar, Philip Mansfield, Sushant Prakash, Bradley Green, Ewa Dominowska, Blaise Aguerre y Arcas, and 12 others. 2023. [Towards expert-level medical question answering with large language models](#). *Preprint*, arXiv:2305.09617.
- Elizabeth C Stadel, Shannon Wiltsey Stirman, Lyle H Ungar, Cody L Boland, H Andrew Schwartz, David B Yaden, João Sedoc, Robert J DeRubeis, Robb Willer, and Johannes C Eichstaedt. 2024. Large language models could change the future of behavioral healthcare: a proposal for responsible development and evaluation. *NPJ Mental Health Research*, 3(1):12.
- Georg Wölflein, Dyke Ferber, Daniel Truhn, Ognjen Arandjelović, and Jakob Nikolas Kather. 2025. [Llm agents making agent tools](#). *Preprint*, arXiv:2502.11705.
- Aiyuan Yang, Bin Xiao, Bingning Wang, Borong Zhang, Ce Bian, Chao Yin, Chenxu Lv, Da Pan, Dian Wang, Dong Yan, Fan Yang, Fei Deng, Feng Wang, Feng Liu, Guangwei Ai, Guosheng Dong, Haizhou Zhao, Hang Xu, Haoze Sun, and 36 others. 2023. [Baichuan 2: Open large-scale language models](#). *Preprint*, arXiv:2309.10305.
- Yunyan Zhang, Zhihong Zhu, and Xian Wu. 2025a. Cmedcalc-bench: A fine-grained benchmark for chinese medical calculations in llm. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 25661–25670.
- Ziyao Zhang, Yanlin Wang, Chong Wang, Jiachi Chen, and Zibin Zheng. 2025b. [Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation](#). *Preprint*, arXiv:2409.20550.
- Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyang Luo, Zhangchi Feng, and Yongqiang Ma. 2024. [Llamafactory: Unified efficient fine-tuning of 100+ language models](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Bangkok, Thailand. Association for Computational Linguistics.
- Aojun Zhou, Ke Wang, Zimu Lu, Weikang Shi, Sichun Luo, Zipeng Qin, Shaoqing Lu, Anya Jia, Linqi Song, Mingjie Zhan, and Hongsheng Li. 2023a. [Solving challenging math word problems using gpt-4 code interpreter with code-based self-verification](#). *Preprint*, arXiv:2308.07921.
- Hongjian Zhou, Fenglin Liu, Boyang Gu, Xinyu Zou, Jinfa Huang, Jing Wu, Yiru Li, Sam S Chen, Peilin Zhou, Junling Liu, and 1 others. 2023b. A survey of large language models in medicine: Progress, application, and challenge. *arXiv preprint arXiv:2311.05112*.

Wenxuan Zhou, Ravi Agrawal, Shujian Zhang, Sathish Reddy Indurthi, Sanqiang Zhao, Kaiqiang Song, Silei Xu, and Chenguang Zhu. 2024. [Wpo: Enhancing rlhf with weighted preference optimization](#). *Preprint*, arXiv:2406.11827.

A Coding Scripts Design details for MedCalc-based Dataset

Table 8 presents representative embedded scripts. We use distinct script designs for equation-based and rule-based calculators.

Equation-based Tasks Equation-based tasks instantiate predefined medical formulas. Single-step calculators are represented by compact expressions, whereas multi-step calculators expose intermediate quantities explicitly. Dosage-conversion scripts define the required conversion constants, and date-based calculators use standard operations from datetime.

Rule-based Tasks Rule-based tasks evaluate calculator criteria against information in the patient note. The script accumulates a score by testing numerical ranges and categorical findings. Numerical inputs are handled with explicit conditional expressions, while categorical variables are assigned from the model interpretation of the clinical description.

We manually verify and execute all script templates. The resulting dataset contains 9,766 training examples and 1,048 test examples, covering 41 calculators grouped into four equation-based and three rule-based task types. Appendix D describes the execution procedure.

B Hyperparameter and Prompts Settings

We follow the inference configuration released with MedCalc-Bench and use LLaMA-Factory defaults as the basis for SFT and preference training.

Data augmentation hyperparameters We set $k = 1,000$ to balance inference cost and the diversity of mined errors. Generating scripts over multiple temperatures produces approximately 1,500 preference pairs.

Training hyperparameters MedCalc-Bench evaluates prompted base models and does not specify SFT or preference-training hyperparameters. We therefore implement both stages with LLaMA-Factory and report the settings in Table 9.

Inference hyperparameters We follow the inference settings in the public MedCalc-Bench implementation. Table 10 lists the values used in our experiments.

Prompts We adapt the code-generation prompts released with MedCalc-Bench. Box E presents the prompt templates.

C Case Study of Minor Error And Substantial Error

Box E uses a GFR calculation to contrast minor and substantial script errors. A minor error changes a local element, such as a coefficient, and can therefore be difficult to distinguish from the reference implementation. A substantial error removes or adds an entire computation step and creates a larger structural difference. wDPO does not require explicit error-category labels; it assigns weights from the model probability gap for each response pair.

We manually categorize generated scripts before and after wDPO as containing *minor* errors (incorrect extracted values, coefficients, or constants, with at most two such errors per script) or *substantial* errors (missing or unnecessary steps, or execution failures). Table 11 shows that the number of minor-error cases decreases from 360 to 312, while substantial-error cases increase from 87 to 144. The reduction of 48 minor-error cases is consistent with improved discrimination of local mistakes, but the simultaneous increase in substantial errors means that the table should not be interpreted as an unqualified reduction in total failures.

D Coding Execution Implementation

Code execution has two stages. **(1) Preprocessing:** Following MedCalc-Bench, we use regular expressions to remove redundant whitespace and line breaks. **(2) Python execution:** We execute the generated script with `exec()` in the controlled evaluation environment and capture its standard output. Calculator descriptions remain in comments, while executable statements perform the calculation and print the response.

E Case Study

Box E presents a complete example in which a fine-tuned model generates and executes code for a medical calculation.

Category	Embedded Coding Script Design (Explanation printings omitted)
Equation-based	Single-step <pre># BMI = weight / (height * height) bmi = 76.0 / ((183.0 / 100) * (183.0 / 100)) bmi = round(bmi, 3) print('The Body Mass Index (BMI) is ', bmi, 'kg/m²')</pre>
	Multi-step <pre># GFR = 175 * creatinine^(-1.154) * age^(-0.203) * race_coefficient * gender_coefficient race_coefficient = 1.212 if 'Black' == 'Black' else 1.0 gender_coefficient = 0.742 if 'Male' == 'Female' else 1.0 gfr = 175 * 10.6**(-1.154) * 49**(-0.203) * race_coefficient * gender_coefficient gfr = round(gfr, 3) print('The MDRD GFR is ', gfr, 'mL/min/ 1.73 m²')</pre>
	Conversion <pre># converted dose = (from mass) * (to reletive mass) / (from reletive mass) reletive_mass = {'Betamethasone IV': 1, 'Cortisone PO': 33.33, 'Dexamethasone IV': 1, 'Dexamethasone PO': 1, 'Hydrocortisone IV': 26.67, 'Hydrocortisone PO': 26.67, 'MethylPrednisolONE IV': 5.33, 'MethylPrednisolONE PO': 5.33, 'PrednisolONE PO': 6.67, 'PredniSONE PO': 6.67, 'Triamcinolone IV': 5.33} converted_dose = 190.936 * reletive_mass['MethylPrednisolONE IV'] / reletive_mass['Hydrocortisone PO'] converted_dose = round(converted_dose, 3) print('The Steroid Conversion is ', converted_dose, 'mg')</pre>
	Date <pre>import datetime # estimated due date = input_date + 40 weeks future_date = datetime.datetime.strptime('11/13/2020', '%m/%d/%Y') + datetime.timedelta(weeks=40) cycle_length_gap = (-1)(23 < 28) datetime.timedelta(days=abs(23 - 28)) estimated_due_date = (future_date + cycle_length_gap).strftime('%m/%d/%Y') print('The Estimated Due Date is ', estimated_due_date, '')</pre>
	Duration <pre>import datetime # estimated gestational age = the number of weeks and days apart today's date is from the patient's last menstrual period date current_date = datetime.datetime.strptime('09/01/2009', '%m/%d/%Y') menstrual_date = datetime.datetime.strptime('05/21/2009', '%m/%d/%Y') weeks = abs(current_date - menstrual_date).days // 7 days = abs(current_date - menstrual_date).days % 7 print('The Estimated Gestational Age is', weeks, 'weeks', days, 'days', '')</pre>
Rule-based	Single-sym. <pre># Centor Score for Strep Pharyngitis criteria: # age, temperature, tonsil swelling, lymph nodes # age thresholds: (45, inf), (3, 15); temperature > 38°C score = 0 score += -1 * (45 <= 48 < float('inf')) + 1 * (3 <= 48 < 15) score += 1 * (38 < ((99.0 - 32) * 5/9) < float('inf')) has_cough_absent = False; score += 1 if has_cough_absent else 0 has_tender_lymph_nodes = False; score += 1 if has_tender_lymph_nodes else 0 has_exudate_swelling_tonsils = False; score += 1 if has_exudate_swelling_tonsils else 0 print('The Centor Score (Modified/McIsaac) for Strep Pharyngitis is', score)</pre>
	Multi-sym. <pre># PERC Rule for Pulmonary Embolism criteria: age, heart_rate, ... # age thresholds are: (50, 'inf') ... # The patient has Hormone use or not score = 0 score += 1 * (50 <= 73 < float('inf')) score += 1 * (100 <= 92.0 < float('inf')) score += 1 * (0 <= 98.0 < 95) has_unilateral_leg_swelling = False; score += 1 if has_unilateral_leg_swelling else 0 has_hemoptysis = False; score += 1 if has_hemoptysis else 0 has_recent_surgery_or_trauma = True; score += 1 if has_recent_surgery_or_trauma else 0 has_pre_pe = False; has_pre_dvt = False; score += 1 if (has_pre_pe or has_pre_dvt) else 0 has_hormonal_use = True; score += 1 if has_hormonal_use else 0 print('The PERC Rule for Pulmonary Embolism is', score)</pre>

Table 8: Examples of embedded scripts for medical calculation. “single-sym.” denotes a criterion containing one symptom, whereas “multi-sym.” denotes a criterion combining multiple symptoms (e.g., has_pre_pe or has_pre_dvt). Explanatory output statements are omitted for brevity.

Box E.1: Prompt Template 1: Direct Inference Prompt

You are a helpful assistant for calculating a score for a given patient note. Please output answer only without any other text. Your output should only contain a JSON dict formatted as {"answer": str(value which is the answer to the question)}.

Here is the patient note:

<Patient Note>

Here is the task:

<Question>

Please directly output the JSON dict formatted as {"answer": str(value which is the answer to the question)}:

Box E.2: Prompt Template 2: CoT Inference Prompt (Zero-shot and One-shot)

You are a helpful assistant for calculating a score for a given patient note. Please think step-by-step to solve the question and then generate the required score. Your output should only contain a JSON dict formatted as

```
{"step_by_step_thinking": str(your_step_by_step_thinking_proccess_to_solve_the_question),  
"answer": str(short_and_direct_answer_of_the_question)}.
```

(Extra prompt for one-shot: start)

Here is an example patient note:

<Example Note>

Here is an example task:

<One Shot Question>

Please directly output the JSON dict formatted as

```
{"step_by_step_thinking": str(your_step_by_step_thinking_proccess_to_solve_the_question), "an-  
swer": str(value which is the answer to the question):json.dumps(example_output)}
```

(Extra prompt for one-shot: end)

Here is the patient note:

<Patient Note>

Here is the task:

<Question>

Please directly output the JSON dict formatted as

```
{"step_by_step_thinking": str(your_step_by_step_thinking_proccess_to_solve_the_question), "an-  
swer": str(short_and_direct_answer_of_the_question)}.
```

Box E.3: Prompt Template 3: Coding Inference Prompt

You are a helpful assistant for calculating a score for a given patient note. Please think step-by-step to solve the question. Do not perform any computations yourself. Do not state a numerical answer. First write code for any equations you are using and then plug in values based on the patient note. Make sure the code prints all of its outputs. Your output should only contain a JSON dict formatted as `{"step_by_step_thinking": str(your_step_by_step_thinking_proccess_to_solve_the_question), "code": str(medical_calculation_code)}`.

(Extra prompt for one-shot: start)

Here is an example patient note:

<Example Note>

Here is an example task:

<One Shot Question>

Please directly output the JSON dict formatted as

```
{"step_by_step_thinking": str(your_step_by_step_thinking_proccess_to_solve_the_question), "code": str(example_medical_calculation_code)}
```

(Extra prompt for one-shot: end)

Here is the patient note:

<Patient Note>

Here is the task:

<Question>

Please directly output the JSON dict formatted as

```
{"step_by_step_thinking": str(your_step_by_step_thinking_proccess_to_solve_the_question), "code": str(medical_calculation_code)}
```

Box E.4: Case Study of Minor Error And Substantial Error on MDRD GFR Coding

Correct:

```
equation = 'GFR=175*creatinine^(-1.154)*age^(-0.203)*race_coeff *gender_coeff'
print('Calculation equation is ', equation)
race_coeff=1.212 if 'White' == 'Black' else 1.0
print('Patient race is White, race coefficient is ', race_coeff)
gender_coeff=0.742 if 'Male' == 'Female' else 1.0
print('Patient gender is Male, gender coefficient is ', gender_coeff)
print('Patient creatinine is 1.4 mg/dL')
print('Patient age is 73 years old')
gfr=175*1.4**(-1.154)*73**(-0.203)*race_coeff*gender_coeff
print('The MDRD GFR is', round(gfr, 3), 'mL/min/1.73 m2')
```

Wrong-minor: gender_coeff value is wrong.

```
equation = 'GFR=175*creatinine^(-1.154)*age^(-0.203)*race_coeff *gender_coeff'
print('Calculation equation is ', equation)
race_coeff=1.212 if 'White' == 'Black' else 1.0
print('Patient race is White, race coefficient is ', race_coeff)
gender_coeff=0.942 if 'Male' == 'Female' else 1.0
print('Patient gender is Male, gender coefficient is ', gender_coeff)
print('Patient creatinine is 1.4 mg/dL')
print('Patient age is 73 years old')
gfr=175*1.4**(-1.154)*73**(-0.203)*race_coeff*gender_coeff
print('The MDRD GFR is', round(gfr, 3), 'mL/min/1.73 m2')
```

Wrong-substantial: missing definition of gender_coefficient = 0.742 if 'Male' == 'Female' else 1.0.)

```
equation = 'GFR=175*creatinine^(-1.154)*age^(-0.203)*race_coeff'
print('Calculation equation is ', equation)
race_coeff=1.212 if 'White' == 'Black' else 1.0
print('Patient race is White, race coefficient is ', race_coeff)
print('Patient creatinine is 1.4 mg/dL')
print('Patient age is 73 years old')
gfr=175*1.4**(-1.154)*73**(-0.203)*race_coeff
print('The MDRD GFR is', round(gfr, 3), 'mL/min/1.73 m2')
```

Box E.5: Case Study: Glomerular Filtration Rate (GFR)

Instruction:

You are a helpful assistant for calculating a score for a given patient note. Please think step-by-step to solve the question. Do not perform any computations yourself. Do not state a numerical answer. First write code for any equations you are using and then plug in values based on the patient note. Make sure the code prints all of its outputs. Your output should only contain a JSON dict formatted as {"step_by_step_thinking": str(your_step_by_step_thinking_process_to_solve_the_question), "code": str(medical_calculation_code)}.

...(Extra prompt for one-shot setting)

Here is the patient note:

A 73-year-old Caucasian male presented with longstanding chronic kidney disease (CKD) G2 related to arterial hypertension stage II. Past medical history: prostate cancer and monoclonal gammopathy of undetermined significance (MGUS) IgG k, rosacea dermatitis, right hand arthritis. Laboratory tests () showed normal complete blood count (CBC), normal serum calcium value, creatinine 1.4 mg/dL, B type natriuretic peptide (BNP) 161 pg/mL, proteinuria 946 mg/24 h, ...(Detailed Patient Notes)... moreover, it could be not uncommon to discover a plasma cell disorder in patients with SAA amyloid, given the increasing incidence of both conditions with chronic inflammatory disease.

Here is the task:

Using the MDRD GFR Equation, what is the patient's Glomerular Filtration Rate (GFR) in terms of mL/min/1.73 m²? If the patient is black, please use the MDRD GFR Equation for Blacks. You should use the patient's medical values and health status when they were first admitted to the hospital prior to any treatment.

Please directly output the JSON dict formatted a

{"step_by_step_thinking": str(your_step_by_step_thinking_process_to_solve_the_question), "code": str(medical_calculation_code)}.

Example Embedded Coding Script (with values embedded):

```
equation = 'GFR=175*creatinine^(-1.154)*age^(-0.203)*race_coeff *gender_coeff'
print('Calculation equation is ', equation)
race_coeff=1.212 if 'White' == 'Black' else 1.0
print('Patient race is White, race coefficient is ', race_coeff)
gender_coeff=0.742 if 'Male' == 'Female' else 1.0
print('Patient gender is Male, gender coefficient is ', gender_coeff)
print('Patient creatinine is 1.4 mg/dL')
print('Patient age is 73 years old')
gfr=175*1.4**(-1.154)*73**(-0.203)*race_coeff*gender_coeff
print('The MDRD GFR is', round(gfr, 3), 'mL/min/1.73 m2')
```

Execution Result:

```
Calculation equation is
GFR=175*creatinine^(-1.154)*age^(-0.203)* race_coeff *gender_coeff
Patient race is White, race coefficient is  1.0
Patient gender is Male, gender coefficient is  1.0
Patient creatinine is 1.4 mg/dL
Patient age is 73 years old
The MDRD GFR is 49.677 mL/min/1.73 m2
```

Box E.6: ICU Dataset: A translated data sample in ICU dataset

Instruction:

Based on the following known information, respond concisely and professionally to the physician's inquiry about the patient's condition. The answer should be in Chinese. Do not fabricate any numerical data values not present in the known content.

EHR:

Known information about the patient's condition:

Vital sign Non-invasive systolic pressure 2024-09-14 00:00:00 123.2mmHg

Vital sign Invasive systolic pressure 2024-09-14 00:21:00 101.4mmHg

Daily vital sign Invasive blood pressure 2024-09-14 00:32:00 153.6mmHg

Vital sign Non-invasive diastolic pressure 2024-09-14 01:19:00 57.2mmHg

Vital sign Invasive diastolic pressure 2024-09-14 01:33:00 73.3mmHg

Daily vital sign Invasive blood pressure 2024-09-14 01:37:00 146.9mmHg

Daily vital sign Non-invasive blood pressure 2024-09-14 01:54:00 91.6mmHg

Vital sign Non-invasive systolic pressure 2024-09-14 02:26:00 117.4mmHg

Please answer the clinician's inquiry about the patient's condition: List the cases where the patient's blood pressure is ≤ 137.2 mmHg and how many times this condition is met.

Example Coding:

```
print('Vital sign Non-invasive systolic pressure 2024-09-14 00:00:00 123.2mmHg' \
if 123.2 <= 137.2 else '')
print('Vital sign Invasive systolic pressure 2024-09-14 00:21:00 101.4mmHg' \
if 101.4 <= 137.2 else '')
print('Daily vital sign Invasive blood pressure 2024-09-14 00:32:00 153.6mmHg' \
if 153.6 <= 137.2 else '')
print('Vital sign Non-invasive diastolic pressure 2024-09-14 01:19:00 57.2mmHg' \
if 57.2 <= 137.2 else '')
print('Vital sign Invasive diastolic pressure 2024-09-14 01:33:00 73.3mmHg' \
if 73.3 <= 137.2 else '')
print('Daily vital sign Invasive blood pressure 2024-09-14 01:37:00 146.9mmHg' \
if 146.9 <= 137.2 else '')
print('Daily vital sign Non-invasive blood pressure 2024-09-14 01:54:00 91.6mmHg' \
if 91.6 <= 137.2 else '')
print('Vital sign Non-invasive systolic pressure 2024-09-14 02:26:00 117.4mmHg' \
if 117.4 <= 137.2 else '')
print('Total', sum(1 for data in [123.2, 101.4, 153.6, 57.2, 73.3, 146.9, \
91.6, 117.4] if data <= 137.2), 'instances of blood pressure<137.2mmHg')
```

Example Outputs:

Vital sign Non-invasive systolic pressure 2024-09-14 00:00:00 123.2mmHg

Vital sign Invasive systolic pressure 2024-09-14 00:21:00 101.4mmHg

Vital sign Non-invasive diastolic pressure 2024-09-14 01:19:00 57.2mmHg

Vital sign Invasive diastolic pressure 2024-09-14 01:33:00 73.3mmHg

Daily vital sign Non-invasive blood pressure 2024-09-14 01:54:00 91.6mmHg

Vital sign Non-invasive systolic pressure 2024-09-14 02:26:00 117.4mmHg

Total 6 instances of blood pressure < 137.2mmHg

Hyperparameters	Value
batch size	4
learning rate	5e-5
epochs	2.0
optimizer	AdamW
cutoff length	2048
data type	bfloat16
preprocessing workers	24
learning rate scheduler	cosine

Table 9: Supervised fine-tuning and preference training hyperparameters.

Hyperparameters	Value
max length	8192/ 4096
temperature	0.0
data type	float16
do sample	False
truncation	True

Table 10: Inference hyperparameters. The maximum length is 8,192 for LLaMA3 and GPT-4 and 4,096 for GPT-3.5, following the MedCalc-Bench configuration.

Method	Minor	Substantial
w/o wDPO	360	87
w/ wDPO	312	144

Table 11: Coding error types before and after wDPO. Minor errors comprise incorrect extracted values, coefficients, or constants, with at most two such errors per script. Substantial errors comprise missing or unnecessary steps and execution failures.

Category	Metric	Unit	Range
Oxygen Saturation	Blood gas analysis oxygen saturation	%	[80, 100]
	Vital sign oxygen saturation (SpO ₂)	%	[80, 100]
Blood Pressure	Vital sign invasive diastolic pressure	mmHg	[40, 90]
	Vital sign non-invasive diastolic pressure	mmHg	[40, 90]
	Vital sign invasive systolic pressure	mmHg	[90, 140]
	Vital sign non-invasive systolic pressure	mmHg	[90, 140]
	Same-day vital sign invasive blood pressure	mmHg	[70, 160]
	Same-day vital sign non-invasive blood pressure	mmHg	[70, 160]
CO ₂ Partial Pressure	Blood gas analysis partial pressure of CO ₂ (pCO ₂)	mmHg	[35, 45]
	Blood gas analysis temperature-corrected partial pressure of CO ₂ (pCO ₂)	mmHg	[35, 45]
Bilirubin	Liver function direct bilirubin	μmol/L	[0, 6.8]
Blood Calcium Concentration	Electrolyte test ionized calcium	mmol/L	[1.1, 1.3]
	Electrolyte test total calcium	mmol/L	[2.1, 2.6]
Serum Calcium	Electrolyte test ionized calcium	mmol/L	[1.1, 1.3]
	Electrolyte test total calcium	mmol/L	[2.1, 2.6]
Blood Potassium	Electrolyte test potassium	mmol/L	[3.5, 5.0]

Table 12: Categories, metrics, units, and clinically plausible ranges in our ICU dataset.